

Manual de Diccionarios en Python

Guía práctica y concisa para dominar dict: desde lo básico hasta patrones de uso profesionales.

1) ¿Qué es un diccionario?

Un **diccionario** (dict) es una colección **mutable**, **indexada por claves** y **ordenada por inserción** (desde Python 3.7+). Se usa para mapear claves (hashables) a valores.

Literal y constructor

```
usuario = {"id": 101, "nombre": "Ana", "activo": True}
vacio = {}
otro = dict(id=102, nombre="Luis", activo=False)
```

Claves válidas: deben ser *hashables* (por ejemplo: str, int, float, tuple de elementos hashables). No puedes usar list o dict como clave.

```
ok = {("A", 1): "par"}
```

```
# no_ok = {[ "A", 1]: "par"} # ✗ TypeError: unhashable type: 'List'
```

Orden: los diccionarios preservan el orden de inserción de elementos (garantizado a partir de Python 3.7).

2) Operaciones esenciales

```
persona = {"nombre": "Sofía", "edad": 29}
```

Lectura

```
persona["nombre"]      # 'Sofía' → KeyError si no existe
persona.get("pais")     # None    → no levanta error
persona.get("pais", "ES") # 'ES'   → valor por defecto
```

Inserción / actualización

```
persona["edad"] = 30
persona["pais"] = "ES"
```

Existencia

```
"edad" in persona      # True
```

Eliminación

```
persona.pop("pais", None) # devuelve y quita; si no existe → None
# persona.pop("pais")     # KeyError si no existe
```

```
# Longitud
len(persona)
```

3) Métodos imprescindibles (cheat sheet)

```
# Obtener con defecto (sin insertar)
d.get(k, default)

# Establecer por defecto (insertando si falta) y devolver el valor
d.setdefault(k, default)

# Actualizar con otro dict o pares clave-valor
d.update(otro_dict)      # o d |= otro_dict (3.9+)

# Eliminar
v = d.pop(k, default)    # quita k y devuelve su valor
k, v = d.popitem()       # quita el último par (LIFO desde 3.7)
d.clear()                # borra todo

# Vistas (se actualizan en vivo)
d.keys(); d.values(); d.items()

# Copias
d.copy()                  # copia superficial
```

NOTA: popitem() extrae el **último** par insertado (LIFO) desde Python 3.7.

4) Recorrido e iteración

```
d = {"a": 1, "b": 2, "c": 3}

for k in d:                # recorre claves
    print(k)

for k in d.keys():         # equivalente
    print(k)

for v in d.values():       # recorre valores
    print(v)

for k, v in d.items():     # pares clave-valor
    print(k, v)
```

Evita modificar (insertar/borrar) mientras iteras el mismo dict. Si necesitas hacerlo, itera sobre una **copia**: `for k in list(d): ....`

5) Comprensiones de diccionarios

```
numeros = [1, 2, 3, 4]
cuadrados = {n: n**2 for n in numeros}
# {1: 1, 2: 4, 3: 9, 4: 16}

# Con filtro
pares = {n: n**2 for n in numeros if n % 2 == 0}
# {2: 4, 4: 16}
```

Transformaciones rápidas

```
# De lista de tuplas a dict
pares = [("es", "España"), ("fr", "Francia")]
mapa = {codigo: nombre for codigo, nombre in pares}

# Re-etiquetar/transformar valores
d = {"a": 1, "b": 2, "c": 3}
triplicado = {k: v * 3 for k, v in d.items()}

# Invertir clave/valor (¡ojo con colisiones!)
inv = {v: k for k, v in d.items()}
```

6) Fusión y unpacking

```
a = {"x": 1, "y": 2}
b = {"y": 999, "z": 3}

# 3.5+ (unpacking)
c = {**a, **b}      # {'x':1, 'y':999, 'z':3}

# 3.9+: operador de unión
c = a | b           # igual resultado, no muta a ni b
a |= b              # muta a → {'x':1, 'y':999, 'z':3}
```

7) Valores por defecto y agrupaciones

7.1) Patrón con dict.setdefault

```
grupo = {}
for alumno, curso in [("Ana", "A"), ("Juan", "A"), ("Luz", "B")]:
    grupo.setdefault(curso, []).append(alumno)
# {'A': ['Ana', 'Juan'], 'B': ['Luz']}
```

7.2) collections.defaultdict

```
from collections import defaultdict
```

```
agrupado = defaultdict(list)
for alumno, curso in [("Ana", "A"), ("Juan", "A"), ("Luz", "B")]:
    agrupado[curso].append(alumno)
# defaultdict(list, {'A': ['Ana', 'Juan'], 'B': ['Luz']})
```

7.3) Contar frecuencias (Counter)

```
from collections import Counter
```

```
palabras = ["data", "ai", "data", "etl"]
conteo = Counter(palabras)      # {'data': 2, 'ai': 1, 'etl': 1}
```

8) Copias y anidación

```
original = {"perfil": {"nombre": "Ana", "skills": ["python", "sql"]}}
```

```
shallow = original.copy()      # copia superficial
```

```
import copy
```

```
deep = copy.deepcopy(original) # copia profunda
```

```
original["perfil"]["skills"].append("spark")
```

```
# afecta a 'shallow' (misma lista interna), pero no a 'deep'
```

Regla: copy() copia solo el primer nivel. Para estructuras anidadas usa deepcopy si necesitas independencia total.

9) Vistas: keys(), values(), items()

Las vistas son **dinámicas**: reflejan cambios posteriores en el diccionario.

```
d = {"a": 1}
ks = d.keys()
d["b"] = 2
list(ks) # ['a', 'b']
```

10) Tipado y protocolos (typing)

- **PEP 585 (3.9+)**: puedes escribir dict[str, int].
- **typing**: Dict[K, V] (legacy), Mapping[K, V] (solo lectura), MutableMapping[K, V] (mutable).
- **TypedDict**: describe diccionarios con forma fija para validación estática.

```
from typing import TypedDict
```

```
class Usuario(TypedDict):
```

```
id: int
nombre: str
activo: bool
```

```
u: Usuario = {"id": 1, "nombre": "Ana", "activo": True}
```

Lectura vs. escritura: anota parámetros como Mapping[str, Any] si solo lees. Eso hace tu API más flexible.

11) Patrones prácticos (cookbook)

11.1) Diccionario como *switch* simple

```
def suma(a, b): return a + b
```

```
def resta(a, b): return a - b
```

```
ops = {
    "+": suma,
    "-": resta,
}
```

```
resultado = ops.get("+", lambda a, b: None)(2, 3) # 5
```

11.2) Normalizar claves (casefold, strip)

```
entrada = {"Nombre": "Ana", "PAIS": "españa"}
normalizado = {k.strip().casefold(): v for k, v in entrada.items()}
# {'nombre': 'Ana', 'pais': 'españa'}
```

11.3) Indexar por una clave de objetos

```
alumnos = [
    {"id": 1, "nombre": "Ana"},
    {"id": 2, "nombre": "Luis"},
]
index = {a["id"]: a for a in alumnos}
# {1: {...}, 2: {...}}
```

11.4) Aplanar anidados (flatten) sencillo

```
data = {"user": {"id": 1, "name": "Ana"}, "meta": {"lang": "es"}}
flat = {f"{outer}.{inner}": v for outer, sub in data.items() for inner, v
in sub.items()}
# {'user.id': 1, 'user.name': 'Ana', 'meta.Lang': 'es'}
```

11.5) JSON $\rightleftharpoons$ dict

```
import json
```

```
payload = {"ok": True, "items": [1, 2, 3]}
```

```
s = json.dumps(payload, ensure_ascii=False)
rec = json.loads(s)  # dict de vuelta
```

12) Rendimiento y complejidad

- **Búsqueda, inserción, borrado:** promedio **O(1)** gracias al *hashing*.
 - Evita hacer dos búsquedas seguidas:
 - En vez de `if k in d: x = d[k]`, usa directamente `x = d.get(k)` o `try: x = d[k] except KeyError: ....`
 - Prefiere comprensiones a bucles con `append` cuando creas nuevos diccionarios.
-

13) Errores comunes y cómo evitarlos

- 1) **KeyError** al acceder: usa `get()` o maneja con `try/except`.
 - 2) **Mutar mientras iteras:** itera sobre `list(d)` si necesitas borrar/insertar.
 - 3) **Claves no hashables:** usa tuplas en vez de listas si necesitas clave compuesta.
 - 4) **Confundir `get()` con `setdefault()`:** `setdefault()` **inserta** si la clave no existe; `get()` no.
 - 5) **Copiar anidados con `copy()`:** recuerda que es superficial.
-

14) Buenas prácticas

- Usa `get()` para lecturas seguras y `pop(k, None)` para eliminaciones seguras.
 - Documenta la forma del diccionario (docstring, `TypedDict` o `pydantic` si aplica).
 - Normaliza claves si vienen de fuentes externas (APIs, CSVs) para evitar duplicados lógicos.
 - Para contadores y agrupaciones, favorece `Counter` y `defaultdict`.
-

15) Ejercicios (con soluciones)

E1) Frecuencias de palabras

Enunciado: dada una lista de palabras, construye un diccionario {palabra: conteo}.

```
palabras = "data etl data ai ai ai".split()
# tu código aquí
```

Solución:

```
from collections import Counter
conteo = Counter(palabras)
```

E2) Agrupar por primera letra

Enunciado: agrupa nombres por inicial en un dict de listas.

```
nombres = ["Ana", "Álvaro", "Luis", "Lola"]
# Nota: usa .casefold() para normalizar acentos/mayúsculas
```

Solución:

```
grupo = {}
for n in nombres:
    inicial = n[0].casefold()
    grupo.setdefault(inicial, []).append(n)
```

E3) Invertir mapa con colisiones

Enunciado: invierte {k:v} a {v:[k,...]} para manejar valores repetidos.

```
d = {"a": 1, "b": 2, "c": 1}
```

Solución:

```
inv = {}
for k, v in d.items():
    inv.setdefault(v, []).append(k)
```

E4) Unir configuraciones con prioridad

Enunciado: une base y override dando prioridad a override.

```
base = {"timeout": 30, "lang": "es"}
override = {"timeout": 10}
```

Solución (3.9+):

```
config = base | override
```

E5) Índice por clave

Enunciado: convierte una lista de dicts de usuarios en un índice por id.

```
users = [  
    {"id": 1, "name": "Ana"},  
    {"id": 2, "name": "Luis"},  
]
```

Solución:

```
index = {u["id"]: u for u in users}
```

E6) Aplanar un dict anidado simple

Enunciado: convierte {"a": {"x": 1}, "b": {"y": 2}} en {"a.x":1, "b.y":2}.

Solución:

```
nested = {"a": {"x": 1}, "b": {"y": 2}}  
flat = {f"{outer}.{inner}": v for outer, sub in nested.items() for inner,  
v in sub.items()}
```

E7) Limpieza de claves

Enunciado: normaliza espacios y mayúsculas en las claves.

```
raw = {" Nombre ": "Ana", "PAIS": "ES"}
```

Solución:

```
clean = {k.strip().casefold(): v for k, v in raw.items()}
```

E8) Eliminación segura

Enunciado: elimina "debug" de cfg sin lanzar error si no existe.

```
cfg = {"timeout": 5, "debug": True}
```

Solución:

```
cfg.pop("debug", None)
```

E9) Validación con TypedDict

Enunciado: define un TypedDict Producto y crea uno válido. **Solución:**

```
from typing import TypedDict  
class Producto(TypedDict):  
    id: int
```

```
nombre: str
precio: float
```

```
p: Producto = {"id": 1, "nombre": "USB", "precio": 9.99}
```

E10) Evitar doble acceso

Enunciado: reescribe este código para evitar doble hash:

```
if k in d:
    x = d[k]
else:
    x = 0
```

Solución:

```
x = d.get(k, 0)
```

16) Referencias rápidas

- Literal: {} | Constructor: dict()
 - Acceso seguro: get(k, default)
 - Inserción con defecto: setdefault(k, default)
 - Unión: a | b y a |= b (3.9+)
 - Vistas dinámicas: keys(), values(), items()
 - Borrado seguro: pop(k, None)
 - Copia superficial: copy() | profunda: copy.deepcopy()
 - Estructuras útiles: defaultdict, Counter, TypedDict
-

Nota de versión

- **3.7+:** orden de inserción garantizado; popitem() → LIFO.
 - **3.9+:** operadores | y |= para unir diccionarios.
-